University of Antwerp
Algorithms and Complexity
Teaching assistant: Tamara Cucumides

Exercises session 2

Decision problems, multitape turing machines, non-deterministic turing machines

---

1 Definitions

Definition 1.1 (Languages and decision problems) *We will now define languages and their corresponding decision problems*

- *Alphabet Σ: a finite set of symbols.*
- *Word w: a finite sequence of symbols from Σ.*
- *Σ^*: The set of all words over the alphabet Σ.*
- *Language L: a set of words.*

We define the decision problem associated to a language L as follows: Given $w \in \Sigma^*$, decide if $w \in L$ (or $w \notin L$).

Definition 1.2 (Multitape Turing Machine) *A deterministic TM with k tapes is defined by $(Q, \Sigma, \Gamma, \delta, q_0, q_{accept}, q_{reject})$, where*

- *Q is a finite set of states*
- *Σ is the input alphabet, with $\vdash, B \notin \Sigma$*
- *Γ is the tape alphabet, with $\Sigma \cup \{\vdash, B\} \subseteq \Gamma$*
- *$q_0 \in Q$ is the initial state*
- *$q_{accept}, q_{reject} \in Q$ are the accepting and rejecting state*
- *δ is a partial function:*

$$\delta \quad : \quad Q \times \Gamma^k \to Q \times \Gamma^k \times \{\leftarrow, \Box, \rightarrow\}^k.$$

  δ is called the transition function.

Definition 1.3 (Non-deterministic Turing Machine) *A non-deterministic turing machine is defined as $(Q, \Sigma, \Gamma, \delta, q_0, q_{accept}, q_{reject})$ where*

- *Q is a finite set of states*
- *Σ is the input alphabet, with $\vdash, B \notin \Sigma$*

- Γ *is the tape alphabet, with* $\Sigma \cup \{\vdash, B\} \subseteq \Gamma$
- $q_0 \in Q$ *is the initial state*
- $q_{accept}, q_{reject} \in Q$ *are the accepting and rejecting states*
- δ *is a* **relation***:*

$$\delta \quad \subseteq \quad Q \times \Gamma \times Q \times \Gamma \times \{\leftarrow, \Box, \rightarrow\}$$

2 Session exercises

1. Reformulate the following problems into a decision problem
   (a) Given a graph, and two special nodes s and t, you want to find a path from s to t
   (b) Given a graph, a node s, and a set of nodes V, what is the shortest possible path that visits all the nodes in V
   (c) You're organising a dinner party. Although you want to invite all of your friends, some of them cant stand each other, and you want a conflict-free dinner.
2. Describe informally (but clearly) a multi-tape turing machine for each of the following problems.
   - $L_1 = \{w : w$ has the same amount of 0s and 1s$\}$
   - $L_2 = \{a^n b^n c^n : n \in \mathbb{N}\}$
   - $L_3 - \{w \cdot w^{-1} : w \in \{0,1\}^*\}$
3. Lets introduce (recall?) boolean formulas
   - **Syntax of boolean formulas**: let's consider a set of variables **Vars**: $\{x_1, \ldots, x_n\}$ and operators $\vee, \wedge, \neg, \rightarrow, \leftrightarrow$, then we define boolean formulas recursively as
     - every variable x_i is a boolean formula
     - if φ and ϕ are boolean formulas, then $(\neg\varphi)$, $(\varphi \wedge \phi)$, $(\varphi \vee \phi)$, $(\varphi \rightarrow \phi)$ and $(\varphi \leftrightarrow \phi)$ are also boolean formulas
   - **Semantics of boolean formulas**: a truth assignment σ maps each variable x_i to either **true** (1) or **false** (0), formally σ : **Vars** $\rightarrow \{0,1\}$. We can evaluate a boolean formula ϕ using a truth assignment σ: we will denote this as $\phi(\sigma)$
     - the evaluation of a variable x_i corresponds to $\sigma(x_i)$
     - if ϕ_1 and ϕ_2 are boolean formulas, then if :
       * $\varphi = (\neg\phi)$, then $\varphi(\sigma) = 1 - \phi(\sigma)$
       * $\varphi = (\phi_1 \wedge \phi_2)$ then $\varphi(\sigma) = \min\{\phi_1(\sigma), \phi_2(\sigma)\}$

  - * $\varphi = (\phi_1 \vee \phi_2)$ then $\varphi(\sigma) = \max\{\phi_1(\sigma), \phi_2(\sigma)\}$
    * $\varphi = (\phi_1 \rightarrow \phi_2)$ then $\varphi(\sigma) = 1$ iff $\phi_1(\sigma) = 0$ or $\phi_2(\sigma) = 1$
    * $\varphi = (\phi_1 \leftrightarrow \phi_2)$ then $\varphi(\sigma) = 1$ iff $\phi_1(\sigma) = \phi_2(\sigma)$
  - **Satisfiability**: we say a boolean formula φ is satisfiable, if there exists a truth assignment σ such that $\varphi(\sigma) = 1$
  - **Conjunctive normal form**: we say a boolean formula is in conjunctive normal form (or CNF) if its a conjunction of disjunctions of literals. A literal is a variable or a negation of a variable. An example:

$$\varphi = (x_1 \vee \neg x_2) \wedge (x_1 \vee x_3)$$

In this exercise we will introduce the SAT problem, that will play an important role in future lessons of this course.

(a) Given a CNF boolean formula φ and an assignment σ for such formula, give a TM that *verifies* whether $\varphi(\sigma) = 1$. What is the running time of such algorithm?

(b) Now lets consider the following decision problem

$$\text{CNF-SAT} = \{\varphi : \varphi \text{ is a satisfiable CNF boolean formula } \}$$

Can we use the idea of the question above to solve this? What would be the running time?

(c) What if we try using non-deterministic turing machines?